

AWS RDS (Relational Database Service)

04-data-engineering/rds

Overview

RDS is AWS managed relational database infrastructure for engines such as PostgreSQL, MySQL, MariaDB, SQL Server, Oracle, and Aurora variants. It handles much of the undifferentiated database operations work while preserving the relational model.

Relational systems remain important in ML platforms because not every dataset is a data lake object. Metadata, transactional application data, user records, and workflow state often fit better in a relational store.

Core Concepts

- RDS manages backups, patching windows, storage, failover features, and monitoring integrations, while the database engine still enforces schemas, indexes, and SQL semantics.
- Important design dimensions include engine choice, Multi Availability Zone deployment, read replicas, storage performance, and connection management.
- Aurora is related but distinct. It offers a cloud optimized relational engine with different performance and scaling characteristics compared with standard RDS engines.

How It Works

- You provision the database into subnets within a VPC, secure it with security groups, and connect applications through standard database drivers.
- Applications issue SQL queries or transactions, while RDS handles storage, backups, failover settings, and many operational hooks in the background.
- Read replicas or analytics exports can separate operational read pressure from write heavy transactional workloads.

AI and ML Relevance

- In ML environments, RDS often stores application ground truth, labeling workflow state, feature metadata, audit records, or model deployment metadata that benefits from strong consistency and joins.
- It can also act as an operational system of record whose data is later extracted into S3 and transformed for training or reporting.

Operational Notes

- Manage schema changes deliberately. Database migrations are production changes and should be treated with the same care as application deployment.
- Watch connection counts carefully when serverless or autoscaling clients are involved. Connection storms are a classic failure mode with relational backends.
- Choose RDS for relational access patterns, not as a default container for any data just because SQL is familiar.

What To Remember

- RDS is about managed operations around relational databases, not a new data model.
- Strong consistency and relational joins are the main reasons to use it in ML systems.
- If access patterns are simple key based lookups at very high scale, DynamoDB may be a better fit than forcing the problem into SQL.